

Atividade 3 - Trabalhando com dados NaN

- 1 - Crie um arquivo no colab.
- 2 - Abra o dataset de vinhos utilizado em aulas passadas.
- 3 - Faça os passos de predição de NaN numérico, codificação de valores categóricos e separação do dataset entre treino e teste.
- 4 - Imprima em PDF o arquivo do colab e entregue a atividade no Classroom.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
```

```
path = 'https://raw.githubusercontent.com/OrlandoIFPR/ml2/main/Dataframes/'
wine_reviews = pd.read_csv(path+ 'winemag-data-130k-v2.csv')
display(wine_reviews.head())
```

Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	taster_name	taster_twitter_handle
0	0	Italy	Aromas include tropical fruit, broom, brimston...	Vulkà Bianco	87	NaN	Sicily & Sardinia	Etna	NaN	Kerin O'Keefe @kerinok
1	1	Portugal	This is ripe and fruity, a wine that is smooth...	Avidagos	87	15.0	Douro	NaN	NaN	Roger Voss @vossn
			Tart and							

Etapa 1 - Diagnostico dos valores ausentes

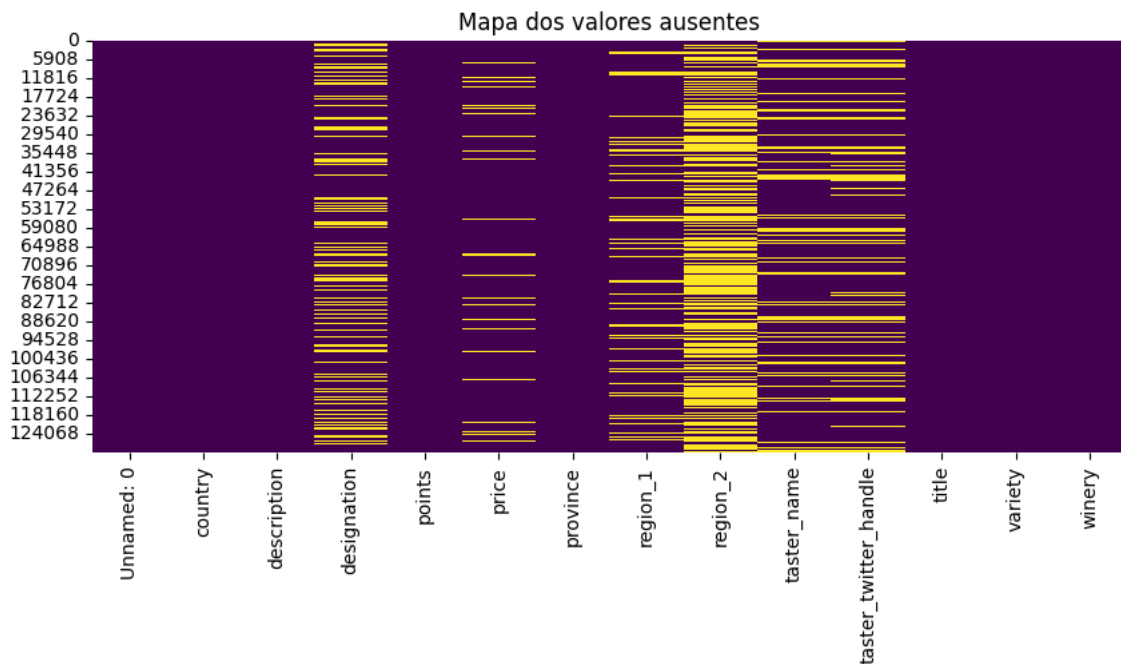
```
nan = pd.DataFrame({
    "nulos": wine_reviews.isnull().sum(),
    "percentual": (wine_reviews.isnull().mean() * 100).round(2)
})
display(nan.sort_values("nulos", ascending=False))

print("Linhas:", wine_reviews.shape[0], " | Colunas:", wine_reviews.shape[1])
```

	nulos	percentual	
region_2	79460	61.14	
designation	37465	28.83	
taster_twitter_handle	31213	24.02	
taster_name	26244	20.19	
region_1	21247	16.35	
price	8996	6.92	
province	63	0.05	
country	63	0.05	
variety	1	0.00	
Unnamed: 0	0	0.00	
points	0	0.00	
description	0	0.00	
title	0	0.00	
winery	0	0.00	
Linhas: 129971 Colunas: 14			

```
plt.figure(figsize=(10, 4))
sns.heatmap(wine_reviews.isnull(), cbar=False, cmap="viridis")
```

```
plt.title("Mapa dos valores ausentes")
plt.show()
```



Leitura: `region_2` (61%) e `designation` (29%) tem NaN demais para serem uteis. A única coluna **numérica** com NaN é `price` (6,9%) - e ela que será **predita**. `country`, `province` e `variety` tem pouquíssimos NaN.

Etapa 2 - Limpeza inicial

Descarto colunas sem valor preditivo (índice, textos livres, identificadores) e as de NaN excessivo; removo as poucas linhas sem `country` / `variety`.

```
df = wine_reviews.drop(columns=[
    "Unnamed: 0",          # apenas o índice do CSV
    "description", "title", # texto livre
    "taster_twitter_handle", # redundante com taster_name
    "designation", "region_2" # NaN em excesso
])

df = df.dropna(subset=["country", "province", "variety"])

print("Formato apos limpeza:", df.shape)
display(df.isnull().sum())
```

Formato apos limpeza: (129907, 8)

	0
country	0
points	0
price	8992
province	0
region_1	21183
taster_name	26243
variety	0
winery	0

dtype: int64

Etapa 3 - Tratamento dos NaN categoricos

`region_1` e `taster_name` não podem ser preditos de forma confiável e não são o alvo da atividade; NaN aqui é informação ("não informado"), então viram uma categoria própria.

```
df["region_1"] = df["region_1"].fillna("Desconhecido")
df["taster_name"] = df["taster_name"].fillna("Desconhecido")
```

```
print("NaN restantes:")
display(df.isnull().sum())
```

NaN restantes:

	0
country	0
points	0
price	8992
province	0
region_1	0
taster_name	0
variety	0
winery	0

dtype: int64

▼ Etapa 4 - Codificacao dos valores categoricos

Os modelos do scikit-learn so aceitam numeros. Verifico primeiro a **cardinalidade** de cada coluna categorica, pois ela define a tecnica.

```
cat_cols = df.select_dtypes(include="object").columns.tolist()
display(df[cat_cols].nunique().sort_values(ascending=False).to_frame("categorias"))
```

	categorias	
winery	16745	
region_1	1230	
variety	701	
province	425	
country	43	
taster_name	20	

Decisao: `winery` (16 mil), `region_1` (1,2 mil) e `variety` (700) tem cardinalidade altissima. One-Hot criaria milhares de colunas esparsas, entao uso **OrdinalEncoder** (1 coluna por atributo), adequado a modelos baseados em arvore, que serao usados na predicao do preco.

Abaixo, um **One-Hot** aplicado a `taster_name` (baixa cardinalidade), so para comparacao das duas tecnicas.

```
from sklearn.preprocessing import OrdinalEncoder

df_enc = df.copy()

encoder = OrdinalEncoder()
df_enc[cat_cols] = encoder.fit_transform(df_enc[cat_cols])

display(df_enc.head())
df_enc.dtypes
```

	country	points	price	province	region_1	taster_name	variety	winery
0	22.0	87	NaN	331.0	425.0	10.0	684.0	11602.0
1	31.0	87	15.0	108.0	389.0	16.0	447.0	12949.0
2	40.0	87	14.0	268.0	1218.0	15.0	433.0	13011.0
3	40.0	87	13.0	218.0	550.0	0.0	476.0	14380.0
4	40.0	87	65.0	268.0	1218.0	15.0	437.0	14610.0

0

country	float64
points	int64
price	float64
province	float64
region_1	float64
taster_name	float64
variety	float64
winery	float64

dtype: object

```
# Comparacao: One-Hot em uma coluna de baixa cardinalidade
exemplo_onehot = pd.get_dummies(df["taster_name"], prefix="taster")
print("Colunas geradas pelo One-Hot:", exemplo_onehot.shape[1])
display(exemplo_onehot.head())
```

Colunas geradas pelo One-Hot: 20

	taster_Alexander Peartree	taster_Anna Lee C. Iijima	taster_Anne Kriebiehl MW	taster_Carrie Dykes	taster_Christina Pickard	taster_Desconhecido	taster_Fiona Adams	taster_Je Jenss
0	False	False	False	False	False	False	False	Fal
1	False	False	False	False	False	False	False	Fal
2	False	False	False	False	False	False	False	Fal
3	True	False	False	False	False	False	False	Fal
4	False	False	False	False	False	False	False	Fal

▼ Etapa 5 - Predicao do NaN numerico (price)

Em vez de preencher com a media (que achata a distribuicao), treino um **RandomForestRegressor** nas linhas que tem preco e uso o modelo para prever as que nao tem.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

com_preco = df_enc[df_enc["price"].notna()]
sem_preco = df_enc[df_enc["price"].isna()]

print("Com preco:", com_preco.shape[0], "| Sem preco:", sem_preco.shape[0])

X_p = com_preco.drop(columns=["price"])
y_p = com_preco["price"]
```

Com preco: 120915 | Sem preco: 8992

```
# Validacao do imputador antes de usa-lo
Xp_tr, Xp_te, yp_tr, yp_te = train_test_split(X_p, y_p, test_size=0.2, random_state=42)

imputador = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
imputador.fit(Xp_tr, yp_tr)

pred = imputador.predict(Xp_te)
print("MAE :", round(mean_absolute_error(yp_te, pred), 2))
print("R2 :", round(r2_score(yp_te, pred), 3))
```

```
print("MAE da media (baseline):",
      round(mean_absolute_error(yp_te, [yp_tr.mean()] * len(yp_te)), 2))
```

```
MAE : 11.06
R2 : 0.357
MAE da media (baseline): 20.1
```

Leitura: o erro do modelo é bem menor que o da média, ou seja, `points`, `country`, `variety` e `winery` realmente carregam informação sobre o preço. O imputador está validado.

```
# Treino final com todos os dados conhecidos e preenchimento dos NaN
imputador.fit(X_p, y_p)
precos_preditos = imputador.predict(sem_preco.drop(columns=["price"]))

df_enc.loc[df_enc["price"].isna(), "price"] = precos_preditos

print("NaN restantes em price:", df_enc["price"].isna().sum())
print("Total de NaN no dataset:", df_enc.isna().sum().sum())
```

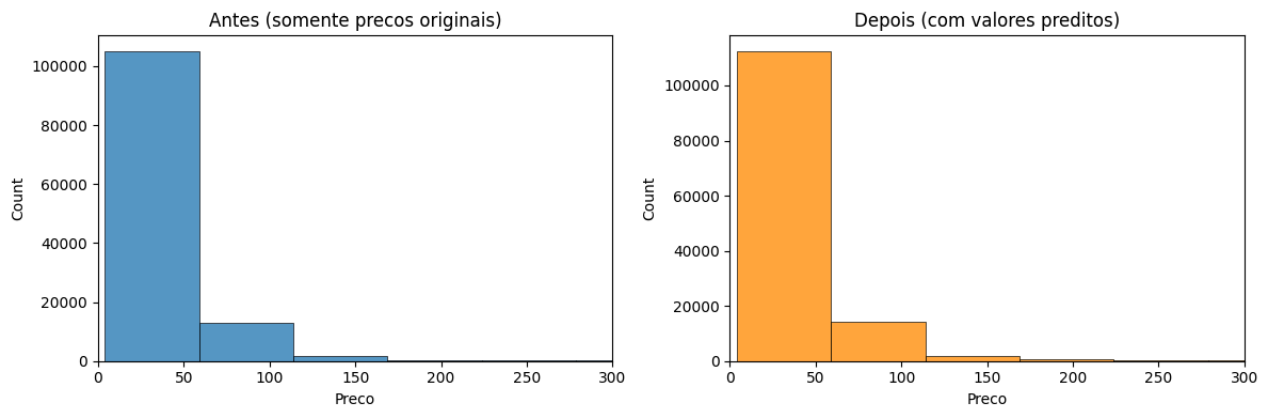
```
NaN restantes em price: 0
Total de NaN no dataset: 0
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(df["price"].dropna(), bins=60, ax=ax[0])
ax[0].set(title="Antes (somente precos originais)", xlim=(0, 300), xlabel="Preco")

sns.histplot(df_enc["price"], bins=60, ax=ax[1], color="darkorange")
ax[1].set(title="Depois (com valores preditos)", xlim=(0, 300), xlabel="Preco")

plt.tight_layout()
plt.show()
```



Leitura: a forma da distribuição (assimétrica à direita) foi preservada. Um `fillna(media)` teria criado um pico artificial em 35, distorcendo a variável.

▼ Etapa 6 - Separação entre treino e teste

Alvo: `points` (nota do vinho). Atributos: todo o restante já codificado e sem NaN.

80% treino / 20% teste

```
X = df_enc.drop(columns=["points"])
y = df_enc["points"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Treino:", X_train.shape, y_train.shape)
print("Teste :", X_test.shape, y_test.shape)
print("Proporção de teste:", round(len(X_test) / len(X) * 100, 1), "%")
```

```
Treino: (103925, 7) (103925,)
Teste : (25982, 7) (25982,)
```

Proporcao de teste: 20.0 %

```
display(X_train.head())
print("Media de points - treino:", round(y_train.mean(), 2),
      "| teste:", round(y_test.mean(), 2))
```

	country	price	province	region_1	taster_name	variety	winery	
17869		22.0	14.0	68.0	1093.0	10.0	69.0	7122.0
50132		15.0	10.0	183.0	1159.0	5.0	80.0	9995.0
35920		40.0	48.0	51.0	912.0	19.0	696.0	888.0
58109		22.0	19.0	374.0	209.0	10.0	470.0	14880.0
119940		40.0	40.0	51.0	36.0	19.0	437.0	12330.0

Media de points - treino: 88.45 | teste: 88.45

Dataset final sem nenhum NaN e inteiramente numerico.